

```

import static org.junit.Assert.assertEquals;

import java.util.Comparator;

import org.junit.Test;

import components.sortingmachine.SortingMachine;

/**
 * JUnit test fixture for {@code SortingMachine<String>}s constructor and
 * kernel methods.
 *
 * @author cl_c2231am03 (Tyler zhuangzhuang He & Meng Zhang)
 */
public abstract class SortingMachineTest {

    /**
     * Invokes the appropriate {@code SortingMachine} constructor for the
     * implementation under test and returns the result.
     *
     * @param order
     *         the {@code Comparator} defining the order for {@code String}
     * @return the new {@code SortingMachine}
     * @requires IS_TOTAL_PREORDER([relation computed by order.compare method])
     * @ensures constructorTest = (true, order, {})
     */
    protected abstract SortingMachine<String> constructorTest(
        Comparator<String> order);

    /**
     * Invokes the appropriate {@code SortingMachine} constructor for the
     * reference implementation and returns the result.
     *
     * @param order
     *         the {@code Comparator} defining the order for {@code String}
     * @return the new {@code SortingMachine}
     * @requires IS_TOTAL_PREORDER([relation computed by order.compare method])
     * @ensures constructorRef = (true, order, {})
     */
    protected abstract SortingMachine<String> constructorRef(
        Comparator<String> order);

    /**
     *
     * Creates and returns a {@code SortingMachine<String>} of the
     * implementation under test type with the given entries and mode.
     *
     * @param order
     *         the {@code Comparator} defining the order for {@code String}
     * @param insertionMode
     *         flag indicating the machine mode
     * @param args
     *         the entries for the {@code SortingMachine}
     * @return the constructed {@code SortingMachine}
     * @requires IS_TOTAL_PREORDER([relation computed by order.compare method])
     * @ensures <pre>
     * createFromArgsTest = (insertionMode, order, [multiset of entries in args])
     * </pre>
     */
    private SortingMachine<String> createFromArgsTest(Comparator<String> order,
        boolean insertionMode, String... args) {
        SortingMachine<String> sm = this.constructorTest(order);
        for (int i = 0; i < args.length; i++) {
            sm.add(args[i]);
        }
        if (!insertionMode) {
            sm.changeToExtractionMode();
        }
    }
}

```

```

        return sm;
    }

    /**
     *
     * Creates and returns a {@code SortingMachine<String>} of the reference
     * implementation type with the given entries and mode.
     *
     * @param order
     *         the {@code Comparator} defining the order for {@code String}
     * @param insertionMode
     *         flag indicating the machine mode
     * @param args
     *         the entries for the {@code SortingMachine}
     * @return the constructed {@code SortingMachine}
     * @requires IS_TOTAL_PREORDER([relation computed by order.compare method])
     * @ensures <pre>
     * createFromArgsRef = (insertionMode, order, [multiset of entries in args])
     * </pre>
     */
    private SortingMachine<String> createFromArgsRef(Comparator<String> order,
        boolean insertionMode, String... args) {
        SortingMachine<String> sm = this.constructorRef(order);
        for (int i = 0; i < args.length; i++) {
            sm.add(args[i]);
        }
        if (!insertionMode) {
            sm.changeToExtractionMode();
        }
        return sm;
    }

    /**
     * Comparator<String> implementation to be used in all test cases. Compare
     * {@code String}s in lexicographic order.
     */
    private static class StringLT implements Comparator<String> {

        @Override
        public int compare(String s1, String s2) {
            return s1.compareToIgnoreCase(s2);
        }

    }

    /**
     * Comparator instance to be used in all test cases.
     */
    private static final StringLT ORDER = new StringLT();

    /**
     * Sample test cases.
     */

    @Test
    public final void testConstructor() {
        SortingMachine<String> m = this.constructorTest(ORDER);
        SortingMachine<String> mExpected = this.constructorRef(ORDER);
        assertEquals(mExpected, m);
    }

    @Test
    public final void testAddEmpty() {
        SortingMachine<String> m = this.createFromArgsTest(ORDER, true);
        SortingMachine<String> mExpected = this.createFromArgsRef(ORDER, true,
            "green");
        m.add("green");
        assertEquals(mExpected, m);
    }

```

```
/*
 * Constructor tests.
 */

@Test
public final void testConstructorIsInInsertionMode() {
    SortingMachine<String> m = this.constructorTest(ORDER);
    assertEquals(true, m.isInInsertionMode());
}

@Test
public final void testConstructorOrder() {
    SortingMachine<String> m = this.constructorTest(ORDER);
    assertEquals(ORDER, m.order());
}

@Test
public final void testConstructorSize() {
    SortingMachine<String> m = this.constructorTest(ORDER);
    assertEquals(0, m.size());
}

/*
 * add tests.
 */

@Test
public final void testAddNonEmpty() {
    SortingMachine<String> m = this.createFromArgsTest(ORDER, true, "red",
        "blue");
    SortingMachine<String> mExpected = this.createFromArgsRef(ORDER, true,
        "red", "blue", "green");
    m.add("green");
    assertEquals(mExpected, m);
}

@Test
public final void testAddDuplicate() {
    SortingMachine<String> m = this.createFromArgsTest(ORDER, true, "red",
        "blue", "red");
    SortingMachine<String> mExpected = this.createFromArgsRef(ORDER, true,
        "red", "blue", "red", "red");
    m.add("red");
    assertEquals(mExpected, m);
}

/*
 * changeToExtractionMode tests.
 */

@Test
public final void testChangeToExtractionModeEmpty() {
    SortingMachine<String> m = this.createFromArgsTest(ORDER, true);
    SortingMachine<String> mExpected = this.createFromArgsRef(ORDER, false);
    m.changeToExtractionMode();
    assertEquals(mExpected, m);
}

@Test
public final void testChangeToExtractionModeOneEntry() {
    SortingMachine<String> m = this.createFromArgsTest(ORDER, true, "red");
    SortingMachine<String> mExpected = this.createFromArgsRef(ORDER, false,
        "red");
    m.changeToExtractionMode();
    assertEquals(mExpected, m);
}

@Test
```

```

public final void testChangeToExtractionModeManyEntries() {
    SortingMachine<String> m = this.createFromArgsTest(ORDER, true,
        "yellow", "blue", "green", "red", "orange");
    SortingMachine<String> mExpected = this.createFromArgsRef(ORDER, false,
        "yellow", "blue", "green", "red", "orange");
    m.changeToExtractionMode();
    assertEquals(mExpected, m);
}

@Test
public final void testChangeToExtractionModeWithDuplicatesAndCase() {
    SortingMachine<String> m = this.createFromArgsTest(ORDER, true, "Red",
        "blue", "red", "BLUE", "green");
    SortingMachine<String> mExpected = this.createFromArgsRef(ORDER, false,
        "Red", "blue", "red", "BLUE", "green");
    m.changeToExtractionMode();
    assertEquals(mExpected, m);
}

@Test
public final void testChangeToExtractionModeHeapifyComplexTree() {
    SortingMachine<String> m = this.createFromArgsTest(ORDER, true,
        "mango", "apple", "kiwi", "banana", "pear", "grape", "fig",
        "date", "cherry", "orange", "lemon");
    m.changeToExtractionMode();
    assertEquals("apple", m.removeFirst());
    assertEquals("banana", m.removeFirst());
    assertEquals("cherry", m.removeFirst());
    assertEquals("date", m.removeFirst());
    assertEquals("fig", m.removeFirst());
    assertEquals("grape", m.removeFirst());
    assertEquals("kiwi", m.removeFirst());
    assertEquals("lemon", m.removeFirst());
    assertEquals("mango", m.removeFirst());
    assertEquals("orange", m.removeFirst());
    assertEquals("pear", m.removeFirst());
    assertEquals(0, m.size());
}

/*
 * removeFirst tests.
 */

@Test
public final void testRemoveFirstOneEntry() {
    SortingMachine<String> m = this.createFromArgsTest(ORDER, false, "red");
    SortingMachine<String> mExpected = this.createFromArgsRef(ORDER, false);
    String first = m.removeFirst();
    assertEquals("red", first);
    assertEquals(mExpected, m);
}

@Test
public final void testRemoveFirstManyEntries() {
    SortingMachine<String> m = this.createFromArgsTest(ORDER, false,
        "yellow", "blue", "green", "red", "orange");
    SortingMachine<String> mExpected = this.createFromArgsRef(ORDER, false,
        "yellow", "green", "red", "orange");
    String first = m.removeFirst();
    assertEquals("blue", first);
    assertEquals(mExpected, m);
}

@Test
public final void testRemoveFirstAlreadySortedEntries() {
    SortingMachine<String> m = this.createFromArgsTest(ORDER, false, "blue",
        "green", "orange", "red", "yellow");
    SortingMachine<String> mExpected = this.createFromArgsRef(ORDER, false,
        "green", "orange", "red", "yellow");

```

```
String first = m.removeFirst();
assertEquals("blue", first);
assertEquals(mExpected, m);
}

@Test
public final void testRemoveFirstReverseSortedEntries() {
    SortingMachine<String> m = this.createFromArgsTest(ORDER, false,
        "yellow", "red", "orange", "green", "blue");
    SortingMachine<String> mExpected = this.createFromArgsRef(ORDER, false,
        "yellow", "red", "orange", "green");
    String first = m.removeFirst();
    assertEquals("blue", first);
    assertEquals(mExpected, m);
}

@Test
public final void testRemoveFirstDuplicateSmallest() {
    SortingMachine<String> m = this.createFromArgsTest(ORDER, false, "blue",
        "red", "blue", "green");
    SortingMachine<String> mExpected = this.createFromArgsRef(ORDER, false,
        "red", "blue", "green");
    String first = m.removeFirst();
    assertEquals("blue", first);
    assertEquals(mExpected, m);
}

@Test
public final void testRemoveFirstUntilEmpty() {
    SortingMachine<String> m = this.createFromArgsTest(ORDER, false,
        "yellow", "blue", "green", "red", "orange");
    SortingMachine<String> mExpected = this.createFromArgsRef(ORDER, false);
    assertEquals("blue", m.removeFirst());
    assertEquals("green", m.removeFirst());
    assertEquals("orange", m.removeFirst());
    assertEquals("red", m.removeFirst());
    assertEquals("yellow", m.removeFirst());
    assertEquals(mExpected, m);
}

@Test
public final void testRemoveFirstSiftDownChoosesLeftChild() {
    SortingMachine<String> m = this.createFromArgsTest(ORDER, false,
        "a", "b", "c", "d", "e", "f", "z");
    assertEquals("a", m.removeFirst());
    assertEquals("b", m.removeFirst());
    assertEquals("c", m.removeFirst());
}

@Test
public final void testRemoveFirstSiftDownChoosesRightChild() {
    SortingMachine<String> m = this.createFromArgsTest(ORDER, false,
        "a", "c", "b", "d", "e", "f", "z");
    assertEquals("a", m.removeFirst());
    assertEquals("b", m.removeFirst());
    assertEquals("c", m.removeFirst());
}

@Test
public final void testRemoveFirstSiftDownRepeatedly() {
    SortingMachine<String> m = this.createFromArgsTest(ORDER, false,
        "i", "h", "g", "f", "e", "d", "c", "b", "a");
    assertEquals("a", m.removeFirst());
    assertEquals("b", m.removeFirst());
    assertEquals("c", m.removeFirst());
    assertEquals("d", m.removeFirst());
    assertEquals("e", m.removeFirst());
    assertEquals(4, m.size());
}
```

```
/*
 * isInInsertionMode tests.
 */

@Test
public final void testIsInInsertionModeAfterAdd() {
    SortingMachine<String> m = this.createFromArgsTest(ORDER, true, "red",
        "blue");
    assertEquals(true, m.isInInsertionMode());
}

@Test
public final void testIsInInsertionModeAfterChangeToExtractionMode() {
    SortingMachine<String> m = this.createFromArgsTest(ORDER, false, "red",
        "blue");
    assertEquals(false, m.isInInsertionMode());
}

/*
 * order tests.
 */

@Test
public final void testOrderInInsertionMode() {
    SortingMachine<String> m = this.createFromArgsTest(ORDER, true, "red",
        "blue");
    assertEquals(ORDER, m.order());
}

@Test
public final void testOrderInExtractionMode() {
    SortingMachine<String> m = this.createFromArgsTest(ORDER, false, "red",
        "blue");
    assertEquals(ORDER, m.order());
}

/*
 * size tests.
 */

@Test
public final void testSizeInsertionModeEmpty() {
    SortingMachine<String> m = this.createFromArgsTest(ORDER, true);
    assertEquals(0, m.size());
}

@Test
public final void testSizeInsertionModeNonEmpty() {
    SortingMachine<String> m = this.createFromArgsTest(ORDER, true, "red",
        "blue", "green");
    assertEquals(3, m.size());
}

@Test
public final void testSizeExtractionModeEmpty() {
    SortingMachine<String> m = this.createFromArgsTest(ORDER, false);
    assertEquals(0, m.size());
}

@Test
public final void testSizeExtractionModeNonEmpty() {
    SortingMachine<String> m = this.createFromArgsTest(ORDER, false, "red",
        "blue", "green");
    assertEquals(3, m.size());
}

@Test
public final void testSizeAfterRemoveFirst() {
```

```
    SortingMachine<String> m = this.createFromArgsTest(ORDER, false, "red",  
        "blue", "green");  
    m.removeFirst();  
    assertEquals(2, m.size());  
}  
  
}
```